

AI as Infrastructure: The Three Paths Forward in an AI-Native World

In the most recent years of society, AI has been extending into almost every domain of life. I can attest to this because I currently work for a school district where I build AI-powered applications, including the customized HCISD AI Application. My current work involves building a chrome extension to automate browser tasks for my administration using the Google Computer Use 2.5 model that just came out on October 6th. Through this hands-on work, I've come to realize something fundamental. AI is no longer just a tool we use. I think it's becoming the standard just like the internet in the '90s. And I believe this shift is creating three career paths that will define how we work with technology for the next coming years.

Look, AI is never going away. Whether we like it or not, it's being embedded in almost everything we do, especially with the rise of AI agents. The future divides into three career paths. First one is building WITH AI as developers using agentic systems. Second one is building THE AI itself as researchers and ML engineers. Finally, the third one is overseeing AI as product managers. My position is simple, you either get good at using AI, or you will be outperformed by the person who knows how to use AI extremely well.

Deploying a production application (like what we did for the project) all with agentic AI systems already tells you this isn't just autocomplete. A 2024 study in Communications of the ACM found that AI pair-programming tools like GitHub Copilot have a huge impact on developer productivity, with benefits spanning task time, product quality, cognitive load, and enjoyment. When I was using Claude Code, I connected various external MCP's. The Playwright MCP could navigate pages, capture screenshots, monitor console errors, and literally see what I saw. The Firebase MCP managed the backend and authentication configuration. The Vercel MCP handled deployments and environment variables. Claude wrote code, saw compile errors via the background dev server output that it ran, fixed TypeScript issues, monitored console errors via Playwright MCP, iterated until resolved, and verified UI all without my manual intervention. This is what infrastructure truly looks like.

This brings me to the core of my position, there's going to be three career paths emerging in an AI-driven world. Building WITH AI is where I currently work, developers using AI agents, Claude Code, Gemini CLI, Codex, Cursor CLI, Copilot CLI, etc. My HCISD AI Application, the Akua project, and the Chrome extension using Google's Computer Use model fall here. The key skill isn't just coding, it's essentially orchestrating AI systems, structuring context, and validating AI-generated solutions. I do believe people who master agentic coding will advance further than those who don't. Building THE AI is where my research interests are. I think ML engineers, researchers, and systems architects will be working on the algorithms, LLMs, agentic systems etc. For example, OpenAI, Google, Anthropic, Grok, Deepseek, and other research labs worldwide. Finally, overseeing AI (which is actually one of my long-term career goals). Product managers bridging technical capabilities and real-world impact. This includes determining how AI systems should be built, deployed, regulated, etc. In fact, a 2025 study of GitHub Copilot deployment across 400+ developers revealed the complexity of effectiveness in large-scale environments. Someone needs to decide when to use AI, how to present its capabilities, the safety of it, and how to handle failures.

However, I want to be clear about something important. Experts in their field should be power users of AI. But beginners have to learn fundamentals first before relying on agentic coding, because experts can tell when AI is wrong, beginners can't. This concern isn't just theoretical. Recent warnings from educators highlight how students are "outsourcing thought to ChatGPT," potentially eroding independent critical thinking skills. I think it's essential that people learn to do things themselves first, form their own ideas, create those ideas, and then transition to AI to augment their work. This applies to all three paths.

With that said, there are questions we still need to answer. My first question, and a question you always asked us, is where do AI agents plateau? My personal experience shows AI agents excel in codebases that are small or when the user can define a task extremely well. But can they handle multiple architectures, scalability, or even security for high-stakes applications in large codebases? A recent MIT study maps these exact roadblocks, revealing that current AI tools struggle with large codebases, proprietary coding conventions, complex tasks, security architecture, etc. There's a difference between writing code that works and designing systems that scale. With that said, it's only a matter of time these get better. My second question is what skills become MORE valuable in an AI-driven world? If AI can generate implementation code effectively, does that make system design, architecture, and domain expertise more valuable? Or does it shift value toward the ability to ask the right questions, structure problems effectively, and validate solutions critically? I suspect both, but the balance isn't yet clear.

AI is transitioning from experimental tool to essential infrastructure. The internet didn't replace businesses; it became a foundation for them. I believe AI is following the same trajectory. The three paths, building WITH AI, building THE AI itself, and overseeing AI systems, aren't mutually exclusive, but the success for future CS students will require competence in at least one. It's only a matter of time before these three paths become the standard way we categorize technical careers. You either get good at using AI, or you will be outperformed by the person who knows how to use AI extremely well. That's the technical reality I've experienced firsthand building production systems with AI agents. And I have biology background. The question isn't whether AI will reshape software development. The question is which path you'll take in that AI future.

References

Zhang, J., Wang, X., Zhang, Y., Sun, H., Wang, K., & Zan, X. (2024). "Large Language Model-Based Agents for Software Engineering: A Survey." arXiv:2409.02977. <https://arxiv.org/abs/2409.02977>

Kalliamvakou, E., Vanderwall, C., Kononenko, O., Damian, D., Lavallée, S., & Storey, M. A. (2024). "Measuring GitHub Copilot's Impact on Productivity." Communications of the ACM, 67(3), 74-82. <https://doi.org/10.1145/3633453>

Vaithilingam, P., Barnett, M., Glassman, E., Eloundou, T., Hicke, P., & Manzano, P. (2025). "Experience with GitHub Copilot for Developer Productivity at Zoominfo." arXiv:2501.13282. <https://arxiv.org/abs/2501.13282>

Andres Gonzales

Hardcastle, K. (2025). "Students are outsourcing thought to ChatGPT — here's why educators should worry, a business professor warns." Business Insider.

<https://www.businessinsider.com/students-outsourcing-thought-chatgpt-educators-worry-2025-9>

MIT CSAIL. (2025). "Can AI really code? Study maps the roadblocks to autonomous software engineering." MIT News.

<https://news.mit.edu/2025/can-ai-really-code-study-maps-roadblocks-to-autonomous-software-engineering-0716>